

S. No.	Date	List of Experiments	Page No.	Signature
1	10/1/26	Study of UNIX Commands & Utilities	1	
2	23/1/26	Process Management using System Calls	8	
3(a)	6/2/26	CPU Scheduling – First Come First Serve (FCFS)	16	
3(b)	6/2/26	CPU Scheduling – Shortest Job First (SJF)	18	
3(c)	13/2/26	CPU Scheduling – Round Robin	20	
3(d)	13/2/26	CPU Scheduling – Priority Scheduling	23	
4	20/2/26	Implementation of Semaphore and Multithreading	25	
5	27/2/26	Deadlock Avoidance using Banker's Algorithm	27	
6	6/3/26	Deadlock Detection Algorithm	30	
7	27/3/26	Memory Allocation – First Fit, Best Fit, Worst Fit	35	
8(a)	10/4/26	FIFO Page Replacement Algorithm	38	
8(b)	10/4/26	LRU Page Replacement Algorithm	40	
8(c)	10/4/26	Optimal Page Replacement Algorithm	43	
9(a)	17/4/26	Disk Scheduling – FCFS	47	
9(b)	17/4/26	Disk Scheduling – SSTF	48	
10(a)	22/4/26	File Allocation – Sequential	50	
10(b)	22/4/26	File Allocation – Indexed	52	
10(c)	22/4/26	File Allocation – Linked	55	

Ex. No: 1

STUDY OF UNIX COMMANDS & UTILITIES

Date:

Aim:

To study and execute Unix commands.

Description:

- Unix is security conscious, and can be used only by those persons who have an account.
- Telnet (Telephone Network) is a Terminal emulator program for TCP/IP networks that enables users to log on to remote servers.
- To logon, type **telnet** server ip address in **run** window.
- User has to authenticate himself by providing username and password. Once verified, a greeting and \$ prompt appears. The shell is now ready to receive commands from the user.
- Options suffixed with a hyphen (–) and arguments are separated by space.

General commands:

Command Function

1.date : It is Used to display the current system date and time.

- **date +%D** : It is used to display date only
- **date +%T** : It is used to display time only
- **date +%Y** : It is used to display the year part of date
- **date +%H** : It is used to display the hour part of time

Syntax: \$ date

Mon Aug 15 15:26:20

2.cal : It is used to indicate Calendar of the current month

- **cal year**: Displays calendar for all months of the specified year
- **cal month year** :Displays calendar for the specified month of the year

Syntax: \$cal July

It displays the month of July

3.who: Login details of all users such as their IP, Terminal No, User name,

- **who am i** : Used to display the login details of the user

Syntax: \$who

stud tty20 Aug 12 14:15

4.tty : Used to display the terminal name

Syntax: \$tty

Now , the system displays /dev/tty1 which indicates, terminal file tty 1 resident in /dev directory.

5.uname: Displays the Operating System

- **uname -r**: Shows version number of the OS (kernel).
- **uname -n** :Displays domain name of the server

Syntax: localhost.localdomain

6.echo "txt": Displays the given text on the screen

- **echo \$HOME**: Displays the user's home directory

Syntax: \$ echo "How are you"

How are you

7.bc :Basic calculator. Press **Ctrl+d** to quit

Syntax: \$ bc

bc 1.06

8. **lp *file*** :Allows the user to spool a job along with others in a print queue.

9. **man *cmdname***: Manual for the given command. Press **q** to exit

10. **history**: To display the commands used by the user since log on.

11. **exit** :Exit from a process. If shell is the only process then logs out

Directory commands:

Command Function

1. **pwd** :Path of the present working directory

Syntax: \$ **pwd**

2. **mkdir filename** A directory is created in the given name under the current directory

Syntax: \$ **mkdir** rushmi

It creates a directory name rushmi in a disk.

3. **mkdir *dir1 dir2***:A number of sub-directories can be created under one stroke

Syntax: \$**mkdir** rucshmi vijay

4. **cd *subdir***: Change Directory. If the *subdir* starts with / then path starts from

Syntax: \$ **cd** rams

It is used to change the current working directory to rams. To check this, type pwd at the system prompt and press enter key. Now, the computer displays /usr/rams

5. **root** (absolute) otherwise from current working directory.

6. **cd** :To switch to the home directory.

7. **cd /:** To switch to the root directory.

8. **rmdir *subdir*** :Removes an empty sub-directory

Syntax: \$ **rmdir** rushmi

It removes the directory named rushmi from a disk

File commands:

Command Function

1. **cat >filename:** To create a file with some contents. To end typing press **Ctrl+d**.

The >symbol means redirecting output to a file. (<for input)

Syntax: \$ **cat > greet**

hi it-a

wishing u the best

2. **cat filename :**Displays the file contents.

Syntax: \$ **cat greet**

hi it-a

wishing u the best

3. **cat >>filename:** Used to append contents to a file

Syntax: \$ **cat >> greet**

bye

\$ cat greet

hi it-a

wishing u the best

bye

4. **cp src des** Copy files to given location. If already exists, it will be overwritten

- \$ cp {source file name } { target filename}
- \$ cp [-r] {source directory name } { target directory name}
- \$ cp [-a] {source directory name } { target directory name}

Where, -r indicates copying directories recursively

-a indicates ask for user confirmation (Y or No)

Syntax: \$ cp 1.c 2.c

here, the contents of 1.c are copied in 2.c. If 2.c doesn't exist, it will be created. If it exists then it is overwritten.

5. **mv *old new*** To rename an existing file or directory. -i option can also be used

- mv *f1 f2 f3 dir* To move a group of files to a directory.
- mv -v *old new* Display name of each file as it is moved.

Syntax: \$ mv 1.c 2.c

Here, the file 1.c is moved from current location to /usr/rams/1.c and the 1.c no longer exists in current working directory.

6. **rm *file*** :Used to delete a file or group of files. -i option can also be used

- rm * To delete all the files in the directory.
- rm -r * Deletes all files and sub-directories
- rm -f * To forcibly remove even write-protected files

Syntax: \$ rm 1.c

the file 1.c will be removed from disk.

7. **Ls** : Lists all files and subdirectories (blue colored) in sorted manner.

- **ls *name*** To check whether a file or directory exists.
- **ls *name**** Short-hand notation to list out filenames of a specific pattern.
- **ls -a** Lists all files including hidden files (files beginning with .)
- **ls -x *dirname*** To have specific listing of a directory.
- **ls -R** Recursive listing of all files in the subdirectories
- **ls -l** Long listing showing file access rights (read/write/execute-**rw**x for user/group/others-**ugo**).

8. **cmp file1 file2** Used to compare two files. Displays nothing if files are identical.

Syntax: \$cmp <file1><file2>

The output is displayed in the form of byte and line number

9. **wc file** : It produces a statistics of lines (**l**), words(**w**), and characters(**c**).

10. **diff** : It locates and reports all differences between two files.

Syntax: \$diff file1 file2

11. **pr** : It prints the contents of a file. It produces output in a single column that contains 66 lines per pages.

Syntax: \$ pr <filename>

12. **banner**:It displays text in form of a banner (large better)

Syntax: \$banner <text>

13. **pg** : It displays contents of the files page by page wise.

Syntax : \$pg <filename>

14. **sleep**: It is used to make some delay for execution in the program.

Syntax: \$ Sleep n

Where, n indicates the number of seconds.

15. **head -n** : It displays first n lines of the file. If n is omitted, the first 10 lines of the files are displayed.

Syntax: \$head -n <filename>

16. **tail -n** : It displays last n lines of the file. If n is omitted, the last 10 lines of the files are displayed.

Syntax: \$tail -n <filename>

17. **cut**: It cuts out selected fields from each line of a file.

Syntax: \$cut -c <list><filename>

Where, <list> indicates the specification for starting and ending column position

e.g. \$cut 5-10 1.c

18. **paste:** It is used to paste the text that was previously used with the cut command.

Syntax: \$paste filename.

Finally to terminate the unix session execute the command **exit** or **logout**.

Result

Thus the study and execution of Unix commands has been completed successfully.

Ex.No.2

PROCESS MANAGEMENT USING SYSTEM CALLS

Date:

a) Program using system call fork()

Aim :

To write the program to create a Child Process using system call fork().

Algorithm :

Step 1 : Declare the variable pid.

Step 2 : Get the pid value using system call fork().

Step 3 : If pid value is less than zero then print as "Fork failed".

Step 4 : Else if pid value is equal to zero include the new process in the system's file using execlp system call.

Step 5 : Else if pid is greater than zero then it is the parent process and it waits till the child completes using the system call wait() Step 6 : Then print "Child complete".

System Calls Used:

1. fork()

Used to create new processes. The new process consists of a copy of the address space of the original process. The value of process id for the child process is zero, whereas the value of process id for the parent is an integer value greater than zero.

Syntax : fork()

2. execlp()

Used after the fork() system call by one of the two processes to replace the process' memory space with a new program. It loads a binary file into memory destroying the memory image of the program containing the execlp system call and starts its execution. The child process overlays its address space with the UNIX command /bin/ls using the execlp system call.

Syntax : execlp()

3. wait()

The parent waits for the child process to complete using the wait system call. The wait system call returns the process identifier of a terminated child, so that the parent can tell which of its possibly many children has terminated.

Syntax : wait(NULL)

4. exit()

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit system call. At that point, the process may return data (output) to its parent process (via the wait system call).

Syntax: exit(0)

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
void main(int argc,char *arg[])
{
int pid;

pid=fork();
if(pid<0)
{
printf("fork failed");
exit(1);
}
else if(pid==0)
{
execlp("whoami","ls",NULL);
exit(0);
}
else
{

printf("\n Process id is -%d\n",getpid());
wait(NULL);
exit(0);
}
}
```

OUTPUT:

```
[cse6@localhost Pgm]$ cc prog4a.c
```

```
The process id is 3175
```

```
[cse6@localhost Pgm]$ ./a.out
```

Result:

Thus the program was executed and verified successfully

b) Program Using System Calls getpid() & getppid()

Aim :

To write the program to implement the system calls getpid() and getppid().

Algorithm :

Step 1 : Declare the variables pid , parent pid , child id and grand child.
Step 2 : Get the child id value using system call fork().
Step 3 : If child id value is less than zero then print as “error at fork() child”.
Step 4 : If child id !=0 then using getpid() system call get the process id.
Step 5 : Print “I am parent” and print the process id.
Step 6 : Get the grand child id value using system call fork().
Step 7 : If the grand child id value is less than zero then print as “error at fork() grand child”.
Step 8 : If the grand child id !=0 then using getpid system call get the process id.
Step 9 : Assign the value of pid to my pid.
Step 10 : Print “I am child” and print the value of my pid.
Step 11 : Get my parent pid value using system call getppid().
Step 12 : Print “My parent’s process id” and its value.
Step 13 : Else print “I am the grand child”.
Step 14 : Get the grand child’s process id using getpid() and print it as “my process id”.
Step 15 : Get the grand child’s parent process id using getppid() and print it as “my parent’s process id”

SYSTEM CALLS USED :

1.getpid()

Each process is identified by its id value. This function is used to get the id value of a particular process.

2.getppid()

Used to get particular process parent’s id value.

3.perror()

Indicate the process error.

Program:

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main( )
{
```

```

int pid;
pid=fork( ); if(pid==-1)
{
perror("fork failed"); exit(0);
}
if(pid==0)
{
printf("\n Child process is under execution");

printf("\n Process id of the child process is %d", getpid()); printf("\n Process id of the parent
process is %d", getppid());
}
else
{
printf("\n Parent process is under execution");

printf("\n Process id of the parent process is %d", getpid()); printf("\n Process id of the child
process in parent is %d", pid()); printf("\n Process id of the parent of parent is %d", getppid());
}
return(0);
}

```

Output:

```

Child process is under execution
Process id of the child process is 9314
Process id of the parent process is 9313 Parent
process is under execution
Process id of the parent process is 9313

```

```

Process id of the child process in parent is 9314 Process id of the parent of parent is 2825

```

Result:

Thus the program was executed and verified successfully

b) Program Using System Call exec()

Aim :

To write the program to implement the system call exec().

Algorithm :

Step 1 : Include the necessary header files.

Step 2 : Print execution of exec system call for the date Unix command.

Step 3 : Execute the execlp function using the appropriate syntax for the Unix command date.

Step 4 : The system date is displayed.

SYSTEM CALL USED :

1.execlp()

Used after the fork() system call by one of the two processes to replace the process' memory space with a new program. It loads a binary file into memory destroying the memory image of the program containing the execlp system call and starts its execution. The child process overlays its address space with the UNIX command /bin/ls using the execlp system call.

Syntax : execlp()

Program:

```
#include<stdio.h>
#include<unistd.h> main( )

{

    printf("\n exec system call");
    printf("displaying the date");
    execlp( "/bin/date", "date", 0);

}
```

Output:

Sat Dec 14 02 : 57 : 38 IST 2010

Result:

Thus the program was executed and verified successfully

d) Program Using System Calls wait() & exit()

Aim :

To write the program to implement the system calls wait() and exit().

Algorithm :

Step 1 : Declare the variables pid and i as integers.

Step 2 : Get the child id value using the system call fork().

Step 3 : If child id value is less than zero then print “fork failed”.

Step 4 : Else if child id value is equal to zero , it is the id value of the child and then start the child process to execute and perform Steps 6 & 7.

Step 5 : Else perform Step 8.

Step 6 : Use a for loop for almost five child processes to be called.

Step 7 : After execution of the for loop then print “child process ends”.

Step 8 : Execute the system call wait() to make the parent to wait for the child process to get over.

Step 9 : Once the child processes are terminated , the parent terminates and hence print “Parent process ends”.

Step 10 : After both the parent and the child processes get terminated it execute the wait() system call to permanently get deleted from the OS.

System Call Used :

1. fork ()

Used to create new process. The new process consists of a copy of the address space of the original process. The value of process id for the child process is zero, whereas the value of process id for the parent is an integer value greater than zero.

Syntax: fork ()

2. wait ()

The parent waits for the child process to complete using the wait system call. The wait system call returns the process identifier of a terminated child, so that the parent can tell which of its possibly many children has terminated.

Syntax: wait (NULL)

3. exit ()

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit system call. At that point, the process may return data (output) to its parent process (via the wait system call).

Syntax: exit(0)

Program:

```
#include<stdio.h>
#include<unistd.h>
main( )
{
    int i, pid;
    pid=fork();
    if(pid== -1)
    {
        perror("fork failed");
        exit(0);
    }

    else if(pid==0)
    {

        printf("\n Child process starts");
        for(i=0; i<5; i++)
        {

            printf("\n Child process %d is called", i);

        }
        printf("\n Child process ends");
    }
    else

    {
        wait(0);
        printf("\n Parent process ends");
    }

    exit(0);
}
```

Output:

Child process starts Child
process 0 is called Child
process 1 is called Child
process 2 is called Child
process 3 is called Child
process 4 is called Child
process ends Parent process
ends

Result:

Thus the program was executed and verified successfully

Ex. No:3a

CPU SCHEDULING FIRST COMEFIRST SERVE

Date:

Aim:

To write a C program to implement the FCFS process scheduling mechanisms.

Algorithm:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Set the waiting of the first process as '0' and its burst time as its turn around time

Step 5: for each process in the Ready Q calculate

(a) Waiting time for process(n)= waiting time of process (n-1) + Burst time of process(n-1)

(b) Turn around time for Process(n)= waiting time of Process(n)+ Burst time for process(n)

Step 6: Calculate

(a) Average waiting time = Total waiting Time / Number of process

(b) Average Turnaround time = Total Turnaround Time / Number of process

Step 7: Stop the process

Program:

```
#include<stdio.h>
int main()
{
    int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
    printf("Enter total number of processes(maximum 20):");
    scanf("%d",&n);
    printf("\nEnter Process Burst Timen");
    for(i=0;i<n;i++)
    {
        printf("P[%d]:",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }
}
```

```

printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++)
{
    tat[i]=bt[i]+wt[i];

    avwt+=wt[i];
    avtat+=tat[i];
    printf("\nP[%d]\t\t\t\t\t",i+1,bt[i],wt[i],tat[i]);
}
avwt/=i;
avtat/=i;
printf("\n\nAverage Waiting Time:%d",avwt);
printf("\n\nAverage Turnaround Time:%d",avtat);
return 0;
}

```

OUTPUT:

```

[csstudent@fedora vajitha]$ ./a.out
Enter the number of processes--5
Enter Burst Time for process 0--20
Enter Burst Time for process 1--3
Enter Burst Time for process 2--6
Enter Burst Time for process 3--4
Enter Burst Time for process 4--5

```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P0	20	0	20
P1	3	20	23
P2	6	23	29
P3	4	29	33
P4	5	33	38

```

Average Waiting Time--21.000000

```

Result:

The program has been executed successfully and output verified

Ex.No:3b

CPU SCHEDULING SHORTEST JOB FIRST

Date:

Aim:

To write a C program to implement the SJF process scheduling mechanisms.

Algorithm:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Start the Ready Q according the shortest Burst time by sorting according to lowest to highest burst time.

Step 5: Set the waiting time of the first process as '0' and its turnaround time as its burst time.

Step 6: For each process in the ready queue, calculate

(a) Waiting time for process(n)= waiting time of process (n-1) + Burst time of process(n-1)

(b) Turn around time for Process(n)= waiting time of Process(n)+ Burst time for process(n)

Step 7: Calculate

(a) Average waiting time = Total waiting Time / Number of process

(b) Average Turnaround time = Total Turnaround Time / Number of process

Step 8: Stop the process

Program:

```
#include<stdio.h>
#include<conio.h>
main()
{
int p[20], bt[20], wt[20], tat[20], i, k, n, temp;
float wtavg, tatavg;
clrscr();
printf("\nEnter the number of processes -- ");
scanf("%d", &n);
for(i=0;i<n;i++)
{
p[i]=i;
printf("Enter Burst Time for Process %d -- ", i);
scanf("%d", &bt[i]);
}
for(i=0;i<n;i++)
```

```

for(k=i+1;k<n;k++)
if(bt[i]>bt[k])
{
temp=bt[i];
bt[i]=bt[k];
bt[k]=temp;
temp=p[i];
p[i]=p[k];
p[k]=temp;
}
wt[0] = wtavg = 0;
tat[0] = tatavg = bt[0];
for(i=1;i<n;i++)
{
wt[i] = wt[i-1] +bt[i-1];
tat[i] = tat[i-1] +bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\n\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND TIME\n");
for(i=0;i<n;i++)
printf("\n\t P%d \t\t %d \t\t %d \t\t %d", p[i], bt[i], wt[i], tat[i]);
printf("\nAverage Waiting Time -- %f", wtavg/n);
printf("\nAverage Turnaround Time -- %f", tatavg/n);
getch();
}

```

Input:

Enter the number of processes -- 4
Enter Burst Time for Process 0 -- 6
Enter Burst Time for Process 1 -- 8
Enter Burst Time for Process 2 -- 7
Enter Burst Time for Process 3 -- 3

Output:

PROCESS BURST TIME WAITING TIME TURNAROUND TIME P3 3 0 3
P0 6 3 9
P2 7 9 16
P1 8 16 24
Average Waiting Time -- 7.000000
Average Turnaround Time -- 13.000000

Result:

The program has been executed successfully and output verified.

Ex.No:3c

CPU SCHEDULING – ROUND ROBIN

Date:

Aim:

To write a C program to implement the Round Robin process scheduling mechanisms.

Algorithm:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue and time quantum (or) time slice

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Calculate the no. of time slices for each process where

No. of time slice for process(n) = burst time process(n)/time slice

Step 5: If the burst time is less than the time slice then the no. of time slices =1.

Step 6: Consider the ready queue is a circular Q, calculate

(a) Waiting time for process(n) = waiting time of process(n-1)+ burst time of process(n-1) + the time difference in getting the CPU from process(n-1)

(b) Turn around time for process(n) = waiting time of process(n) + burst time of process(n)+ the time difference in getting CPU from process(n).

Step 7: Calculate

(a) Average waiting time = Total waiting Time / Number of process

(b) Average Turnaround time = Total Turnaround Time / Number of process

Step 8: Stop the process

Program:

```
#include<stdio.h>
#include<conio.h>
struct process
{
int pn,bt,wt,tat;
}p[10];
void main()
{
int tq,i,j=0,n,b[10],ttat=0;
float awt=0,atat=0;
clrscr();
printf("\n enter no. of process:");
scanf("%d",&n);
```

```

printf("\n enter input:\np.no\tburs tim\n");
for(i=1;i<=n;i++)
{
scanf("%d %d",&p[i].pn,&p[i].bt);
b[i]=p[i].bt;
ttat=ttat+p[i].bt;
}
printf("\n enter time quantum:");
scanf("%d",&tq);
while(j<ttat)
{
for(i=1;i<=n;i++)
{
if(p[i].bt!=0)
{
if(p[i].bt>tq)
{
p[i].bt=p[i].bt-tq;
j=j+tq;
}
else
{
j=j+p[i].bt;
p[i].tat=j;
p[i].wt=p[i].tat-b[i];
atat=atat+p[i].tat;
awt=awt+p[i].wt;
p[i].bt=0;
} } }
printf("\np.no\tburst\twait\tturn\n:");
for(i=1;i<=n;i++)
{
printf("\n%d\t%d\t%d\t%d",p[i].pn,b[i],p[i].wt,p[i].tat); printf("\n");
}
printf("\n avg wait time is %f",awt/n);
printf("\n avg turn time is %f",atat/n);
getch();
}

```

OUTPUT:

enter no. of process:3

enter input:

p.no burs time

1 24

2 3

3 3

enter time quantum:4

p.no burst wait turn:

1 24 6 30

2 3 4 7

3 3 7 10

avg wait time is 5.666667

avg turn time is 15.666667

Result:

The program has been executed successfully and output verified

Ex.No:3d

CPU SCHEDULING – PRIORITY

Date:

Aim:

To write a C program to implement the priority process scheduling mechanisms.

Algorithm:

1. Start the process
2. Get the number of processes to be inserted
3. Get the corresponding priority of processes
4. Sort the processes according to the priority and allocate the one with highest priority to execute first
5. If two process have same priority then FCFS scheduling algorithm is used
6. Calculate the total and average waiting time and turnaround time
7. Display the values
8. Stop the process

PROGRAM:

```
#include<stdio.h>
main()
{
int p[20],bt[20],pri[20], wt[20],tat[20],i, k, n, temp;
float wtavg, tatavg;
clrscr();
printf("Enter the number of processes --- ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
p[i] = i;
printf("Enter the Burst Time & Priority of Process %d --- ",i);
scanf("%d %d",&bt[i], &pri[i]);
}
for(i=0;i<n;i++)
for(k=i+1;k<n;k++)
if(pri[i] > pri[k])
{
temp=p[i];
p[i]=p[k];
p[k]=temp;
temp=bt[i];
bt[i]=bt[k];
bt[k]=temp;
temp=pri[i];
pri[i]=pri[k];
pri[k]=temp;
}
```

```

}
wtavg = wt[0] = 0;
tatavg = tat[0] = bt[0];
for(i=1;i<n;i++)
{
wt[i] = wt[i-1] + bt[i-1];
tat[i] = tat[i-1] + bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\nPROCESS\t\tPRIORITY\tBURST TIME\tWAITING TIME\tTURNAROUND
TIME"); for(i=0;i<n;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d ",p[i],pri[i],bt[i],wt[i],tat[i]);
printf("\nAverage Waiting Time is --- %f",wtavg/n);
printf("\nAverage Turnaround Time is --- %f",tatavg/n);
getch();
}

```

Input:

Enter the number of processes -- 5
Enter the Burst Time & Priority of Process 0 --- 10 3
Enter the Burst Time & Priority of Process 1 --- 1 1
Enter the Burst Time & Priority of Process 2 --- 2 4
Enter the Burst Time & Priority of Process 3 --- 1 5
Enter the Burst Time & Priority of Process 4 --- 5 2

Output:

PROCESS PRIORITY BURST TIME WAITING TIME TURNAROUND TIME 1 1 1 0 1
4 2 5 1 6
0 3 10 6 16
2 4 2 16 18
3 5 1 18 19
Average Waiting Time is --- 8.200000
Average Turnaround Time is --- 12.000000

Result:

Thus the program was executed successfully and hence output verified

Ex.no.4 IMPLEMENTATION OF SEMAPHORE AND MULTI-THREADING

Date:

Aim:

To implement **mutual exclusion using semaphores in threads** and demonstrate synchronization between multiple threads accessing a shared critical section using the **POSIX thread**

Algorithm:

1. Start the program.
2. Declare a semaphore variable for synchronization.
3. Initialize the semaphore value to 1 using `sem_init()` to allow only one thread to enter the critical section at a time.
4. Create two threads using `pthread_create()`.
5. Each thread executes the following steps:
 - Call `sem_wait()` to acquire the semaphore lock.
 - Enter the critical section and display a message.
 - Perform some operation (`sleep(3)` to simulate processing).
 - Display completion message.
 - Call `sem_post()` to release the semaphore lock.
6. Main thread waits for both threads to complete using `pthread_join()`.
7. Destroy the semaphore using `sem_destroy()`.
8. Stop the program.

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex; // Declare a semaphore variable

void* thread_function(void* arg) {
    // Wait (lock) for the semaphore
    sem_wait(&mutex);
    printf("\nEntered Critical Section..\n");

    // Critical section (shared resource access)
    sleep(3); // Simulate some work

    printf("Completed work and Exiting Critical Section...\n");
    // Signal (unlock) the semaphore
    sem_post(&mutex);
}
```

```

return NULL;
}

int main() {
pthread_t th1, th2;

// Initialize the semaphore to 1 (binary semaphore for mutual exclusion)
// The second argument (0) means the semaphore is shared between threads of the same process
sem_init(&mutex, 0, 1);

// Create the first thread
pthread_create(&th1, NULL, thread_function, NULL);
sleep(1); // Give the first thread time to potentially start first

// Create the second thread
pthread_create(&th2, NULL, thread_function, NULL);

// Join threads with the main thread to wait for their completion
pthread_join(th1, NULL);
pthread_join(th2, NULL);

// Destroy the semaphore to release resources
sem_destroy(&mutex);

return 0;
}

```

Output:

```

Entered Critical Section..
Completed work and Exiting Critical Section...
Entered Critical Section..
Completed work and Exiting Critical Section...

```

Result:

Thus the program was executed successfully and hence output verified

Ex. No. 5 IMPLEMENTATION OF DEADLOCK AVOIDANCE USING BANKER'S ALGORITHM

Date:

Aim:

Simulate Banker's Algorithm for Deadlock Avoidance to find whether the system is in safe state or not.

Algorithm:

1. Start the program.
2. Get the values of resources and processes.
3. Get the avail value.
4. After allocation find the need value.
5. Check whether its possible to allocate.
6. If it is possible then the system is in safe state.
7. Else system is not in safety state.
8. If the new request comes then check that the system is in safety.
9. or not if we allow the request.
10. stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int
    k=0,output[10],d=0,t=0,ins[5],i,avail[5],allocated[10][5],need[10][5],MAX[10][5],pno,P[10],j,rz
    , count=0;
    clrscr();
    printf("\n Enter the number of resources : ");
    scanf("%d", &rz);
    printf("\n enter the max instances of each resources\n");
    for(i=0;i<rz;i++)
    { avail[i]=0;
    printf("%c= ",(i+97));
```

```

scanf("%d",&ins[i]);
}
printf("\n Enter the number of processes : ");
scanf("%d", &pno);
printf("\n Enter the allocation matrix \n    ");

for(i=0;i<rz;i++)
printf(" %c",(i+97));
printf("\n");
for(i=0;i <pno;i++)
{
    P[i]=i;
printf("P[%d] ",P[i]);
for(j=0;j<rz;j++)
{
scanf("%d",&allocated[i][j]);
avail[j]+=allocated[i][j];
}
}

printf("\nEnter the MAX matrix \n    ");
for(i=0;i<rz;i++)
{
    printf(" %c",(i+97));
avail[i]=ins[i]-avail[i];
}
printf("\n");
for(i=0;i <pno;i++)
{
printf("P[%d] ",i);
for(j=0;j<rz;j++)
scanf("%d", &MAX[i][j]);
}

printf("\n");
A: d=-1;
for(i=0;i <pno;i++)
{
    count=0; t=P[i];
for(j=0;j<rz;j++)
{
need[t][j] = MAX[t][j]-allocated[t][j];
if(need[t][j]<=avail[j])
count++;
}
if(count==rz)
{
output[k++]=P[i];
for(j=0;j<rz;j++)

```

```

avail[j]+=allocated[t][j];
}
else
P[++d]=P[i];
}

if(d!=-1)
{ pno=d+1;
goto A;
}
printf("\t <");
for(i=0;i<k;i++)
printf(" P[%d] ",output[i]);
printf(">");
getch();
}

```

Sample Input and Output:

```

Enter the number of resources:3
Enter the max instances of each resources
A=10
B=7
C=7
Enter the number of process:3
A      b      c
P[0]   3      2      1
P[1]   1      1      2
P[2]   4      1      2
Enter the Max matrix
A      b      c
P[0]   4      4      4
P[1]   3      4      5
P[2]   5      2      4
<p[2] p[0] p[1]>

```

Result:

Thus the banker's algorithm has been implemented to check whether the state is safe or not.

Ex.No.6 SIMULATE ALGORITHM FOR DEADLOCK DETECTION

Date:

Aim:

To Simulate Algorithm for Deadlock Detection .

Algorithm:

1. Start
2. Attacking Mutex condition : never grant exclusive access. but this may not be possible for several resources.
3. .Attacking preemption: not something you want to do.
4. Attacking hold and wait condition : make a process hold at the most 1 resource at a time. Make all the requests at the beginning. All or nothing policy. If you feel, retry. eg. 2-phase locking 34.
5. Attacking circular wait: Order all the resources. Make sure that the requests are issued in the correct order so that there are no cycles present in the resource graph. Resources numbered 1 ... n. Resources can be requested only in increasing order. ie. you cannot request a resource whose no is less than any you may be holding.
6. Stop

Program:

```
#include<stdio.h>
#include<conio.h>
int max[100][100];
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("***** Deadlock Detection Algo *****\n");
    input();
    show();
    cal();
    getch();
    return 0;
}
```

```

}
void input()
{
int i,j;
printf("Enter the no of Processes\t");
scanf("%d",&n);
printf("Enter the no of resource instances\t");
scanf("%d",&r);
printf("Enter the Max Matrix\n");
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
scanf("%d",&max[i][j]);
}
}
printf("Enter the Allocation Matrix\n");
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
scanf("%d",&alloc[i][j]);
}
}
printf("Enter the available Resources\n");
for(j=0;j<r;j++)
{
scanf("%d",&avail[j]);
}
}
void show()
{
int i,j;
printf("Process\t Allocation\t Max\t Available\t");
for(i=0;i<n;i++)
{
printf("\nP%d\t ",i+1);
for(j=0;j<r;j++)
{
printf("%d ",alloc[i][j]);
}
printf("\t");
for(j=0;j<r;j++)
{
printf("%d ",max[i][j]);
}
}

```

```

printf("\t");
if(i==0)
{
for(j=0;j<r;j++)
printf("%d ",avail[j]);
}
}
}
void cal()
{
int finish[100],temp,need[100][100],flag=1,k,c1=0;
int dead[100];
int safe[100];
int i,j;
for(i=0;i<n;i++)
{
    finish[i]=0;
}
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
    need[i][j]=max[i][j]-alloc[i][j];
}
}
while(flag)
{
flag=0;
for(i=0;i<n;i++)
{
int c=0;
for(j=0;j<r;j++)
{
if((finish[i]==0)&&(need[i][j]<=avail[j]))
{
c++;
if(c==r)
{
for(k=0;k<r;k++)
{
avail[k]+=alloc[i][j];
finish[i]=1;
flag=1;
}
}
}
}
}
}
//printf("\nP%d",i);

```

```

if(finish[i]==1)
    {
        i=n;
    }
}

}}
j=0;
flag=0;
for(i=0;i<n;i++)
{
    if(finish[i]==0)
    {
        dead[j]=i;
        j++;
        flag=1;
    }
}
if(flag==1)
{
    printf("\n\nSystem is in Deadlock and the Deadlock process are\n");
    for(i=0;i<n;i++)
    {
        printf("P%d\t",dead[i]);
    }
}
else
{
    printf("\nNo Deadlock Occur");
} }

```

Output:

```
*****Deadlock detection*****
Enter the no of processes      3
Enter the o of resource instances      3
Enter the max matrix
3      6      8
4      3      3
3      4      4
Enter the allocation matrix
3      3      3
2      0      3
1      2      4
Enter the available resources
1      2      0
Process      Allocation      Max      Available
P1      3 3 3      3 6 8      1 2 0
P2      2 0 3      4 3 3
P3      1 2 4      3 4 4
System is in deadlock and the process are P0 p1      p2
```

Result:

Thus the Algorithm for Deadlock Detection was implemented.

EX.NO. 7 IMPLEMENT FIRST-FIT, BEST-FIT, AND WORST-FIT MEMORY ALLOCATION STRATEGIES

Date:

Aim:

To implement First Fit, Worst Fit ,and Best Fit Memory Allocation Methods

First Fit Algorithm:

- 1- Input memory blocks with size and processes with size.
- 2- Initialize all memory blocks as free.
- 3- Start by picking each process and check if it can be assigned to current block.
- 4- If size-of-process $\leq$ size-of-block if yes then assign and check for next process.
- 5- If not then keep checking the further blocks.

Best-Fit Algorithm:

1. Declare structures *hole* and *process* to hold information about set of holes and processes respectively.
2. Get number of holes, say *nh*.
3. Get the size of each hole
4. Get number of processes, say *np*.
5. Get the memory requirements for each process.
6. Allocate processes to holes, by examining each hole as follows:
 - a. Sort the holes according to their sizes in ascending order
 - b. If hole size $>$ process size then
 - i. Mark process as allocated to that hole.
 - ii. Decrement hole size by process size.
 - c. Otherwise check the next from the set of sorted hole
7. Print the list of process and their allocated holes or unallocated status.
8. Print the list of holes, their actual and current availability.
9. Stop

Program:

```
#include<stdio.h>
void main()
{
    int i,j,temp,b[10],c[10],arr,n,ch,a;
    printf("\t\t FIRST FIT, BEST FIT, WORST FIT\n");
    printf("Enter the size of no. of blocks:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        printf("Enter the size of %d block:",i);
        scanf("%d",&b[i]);
        c[i]=b[i];
    }
}
```

```

}
printf("\nEnter the size of Arriving block:");
scanf("%d",&arr);
printf("\n1.First fit\n2.Best fit\n3.Worst fit\nEnter your choice:"); scanf("%d",&ch);
switch(ch)
{
case 1:
for(i=1;i<=n;i++)
{
if(b[i]>=arr)
{
printf("\t\t%d",arr);
printf("\nArriving block is allocated to %d block.",i); break;
}
else
printf("%d",b[i]);
continue;
}
break;
case 2:
for(i=1;i<=n;i++) {
if(b[i]>=b[i+1]) {
temp=b[i];
b[i]=b[i+1]; b[i+1]=temp; }
}
for(i=1;i<=n;i++) {
if(b[i]>=arr) {
a=b[i];
break;
}
else
printf("%d",b[i]); continue;
}
for(i=1;i<=n;i++) {
if(c[i]==a)
{
printf("\t\t%d",arr);
printf("\nArriving block is allocated to %d block.",i); }
}
break;
case 3:
for(i=1;i<=n;i++)
{
if(b[i]>=b[i+1])
{
temp=b[i];

```

```

b[i]=b[i+1];
b[i+1]=temp;
}
}
for(i=1;i<n;i++)
printf(" %d",b[i]);
printf("\t%d",arr);
printf("\n Arriving block is allocated to %d block",i); break;
default:
printf("Enter the valid choice:");
}
}
}

```

Output:

FIRST FIT, BEST FIT, WORST FIT

Enter the size of no. of blocks:5

Enter the size of 1 block:100

Enter the size of 2 block:500

Enter the size of 3 block:200

Enter the size of 4 block:300

Enter the size of 5 block:600

Enter the size of Arriving block:212

1.First fit

2.Best fit

3. Worst fit

Enter your choice:1

212

Arriving block is allocated to 2 block.

Result:

Thus the program was executed successfully and hence output verified

Ex.No. 8.a IMPLEMENT PAGE REPLACEMENT ALGORITHMS- FIFO ALGORITHM

Date:

Aim:

To Implement The FIFO Page Replacement Algorithm

Algorithm:

Step 1. Start to traverse the pages.

Step 2. If the memory holds fewer pages, then the capacity else goes to step 5.

Step 3. Push pages in the queue one at a time until the queue reaches its maximum capacity or all page requests are fulfilled.

Step 4. If the current page is present in the memory, do nothing.

Step 5. Else, pop the topmost page from the queue as it was inserted first. Step 6. Replace the topmost page with the current page from the string.

Step 7. Increment the page faults.

Step 8. Stop

Program:

```
#include <stdio.h>
int main()
{
int incomingStream[] = {4, 1, 2, 4, 5};
int pageFaults = 0;
int frames = 3;
int m, n, s, pages;
pages = sizeof(incomingStream)/sizeof(incomingStream[0]);
printf("Incoming \t Frame 1 \t Frame 2 \t Frame 3");
int temp[frames];
for(m = 0; m < frames; m++)
{
temp[m] = -1;
}
for(m = 0; m < pages; m++)
{
s = 0;
for(n = 0; n < frames; n++)
{
if(incomingStream[m] == temp[n])
{
s++;
pageFaults--;
}
}
pageFaults++;
}
```

```

if((pageFaults <= frames) && (s == 0))
{
temp[m] = incomingStream[m];
}
else if(s == 0)
{
temp[(pageFaults - 1) % frames] = incomingStream[m]; }
printf("\n");
printf("%d\t\t\t",incomingStream[m]);
for(n = 0; n < frames; n++)
{
if(temp[n] != -1)
printf(" %d\t\t\t", temp[n]);
else
printf(" - \t\t\t");
}
}
printf("\nTotal Page Faults:\t%d\n", pageFaults); return 0;
}

```

Output:

```

Incoming Frame 1 Frame 2 Frame 3
4 4 - -
1 4 1 -
2 4 1 2
4 4 1 2
5 5 1 2
Total Page Faults: 4

```

Result:

Thus the program was executed successfully and hence output verified

Ex.No: 8.b

LRU PAGE REPLACEMENT ALGORITHM

Date:

Aim:

To write a c program to implement LRU page replacement algorithm

Algorithm:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least recently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

Program:

```
#include<stdio.h>
main()
{
int q[20],p[50],c=0,c1,d,f,i,j,k=0,n,r,t,b[20],c2[20];
printf("Enter no of pages:");
scanf("%d",&n);
printf("Enter the reference string:");
for(i=0;i<n;i++)
scanf("%d",&p[i]);
printf("Enter no of frames:");
scanf("%d",&f);
q[k]=p[k];
printf("\n\t%d\n",q[k]);
c++;
k++;
for(i=1;i<n;i++)
{
c1=0;
for(j=0;j<f;j++) {
if(p[i]!=q[j])
c1++;
}
if(c1==f)
{
c++;
if(k<f)
```

```

{
q[k]=p[i];
k++;
for(j=0;j<k;j++)printf("\t%d",q[j]);printf("\n");
}
else
{
for(r=0;r<f;r++) {
c2[r]=0;
for(j=i-1;j<n;j--){
if(q[r]!=p[j])
c2[r]++;
else
break;
}
}
for(r=0;r<f;r++)b[r]=c2[r];
for(r=0;r<f;r++) {
for(j=r;j<f;j++) {
if(b[r]<b[j])
{
t=b[r];
b[r]=b[j];
b[j]=t;
}
}
}
for(r=0;r<f;r++) {
if(c2[r]==b[0])
q[r]=p[i];
printf("\t%d",q[r]);
}
printf("\n");
}
}
}
printf("\nThe no of page faults is %d",c);
}

```

Output:

Enter no of pages:10

Enter the reference string:7 5 9 4 3 7 9 6 2 1

Enter no of frames:3

7

7 5

7 5 9

4 5 9

4 3 9

4 3 7

9 3 7

9 6 7

9 6 2

1 6 2

The no of page faults is 10

Result:

Thus the program was executed successfully and hence output verified

Ex.No. 8. c**OPTIMAL PAGE REPLACEMENT ALGORITHM****Date:****Aim:**

To write a C program to implement the **Optimal Page Replacement algorithm** and calculate the number of page faults.

Algorithm:

1. Start the program.
2. Input the number of frames and page reference string.
3. Initialize all frames as empty.
4. For each page in the reference string:
 - Check if the page is already in any frame.
5. If present:
 - No page fault, move to next page.
6. If not present:
 - If empty frame exists, place the page.
 - Else:
 - Check future references of all pages in frames.
 - Find the page that will not be used for the longest time.
 - Replace that page with the current page.
7. Increment page fault count whenever replacement occurs.
8. Repeat until all pages are processed.
9. Display total number of page faults.
10. Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
main()
{
int fr[5],i,j,k,t[5],p=1,flag=0,page[25],psz,nf,t1,u[5];
clrscr();
printf("enter the number of frames:");
scanf("%d",&nf);
printf("\n enter the page size");
scanf("%d",&psz);
printf("\n enter the page sequence:");
for(i=1; i<=psz; i++)
scanf("%d",&page[i]);
for(i=1; i<=nf; i++)
fr[i]=-1;
for(i=1; i<=psz; i++)
```

```

{
if(full(fr,nf)==1)
break;
else
{
flag=0;
for(j=1; j<=nf; j++)
{
if(page[i]==fr[j])
{
flag=1;
printf(" \t%d:\t",page[i]); break;
}
}
}
if(flag==0)
{
fr[p]=page[i];
printf(" \t%d:\t",page[i]); p++;
}
for(j=1; j<=nf; j++)
printf(" %d ",fr[j]);
printf("\n");
}
}
p=0;
for(; i<=psz; i++)
{
flag=0;
for(j=1; j<=nf; j++)
{
if(page[i]==fr[j])
{
flag=1;
break;
}
}
if(flag==0)
{
p++;
for(j=1; j<=nf; j++)
{
for(k=i+1; k<=psz; k++)
{
if(fr[j]==page[k])
{
u[j]=k;

```

```

break;
}
else
u[j]=21;
}
}
for(j=1; j<=nf; j++)
t[j]=u[j];
for(j=1; j<=nf; j++)
{
for(k=j+1; k<=nf; k++)
{
if(t[j]<t[k])
{
t1=t[j];
t[j]=t[k];
t[k]=t1;
}
}
}
for(j=1; j<=nf; j++)
{
if(t[1]==u[j])
{
fr[j]=page[i];
u[j]=i;
}
}
printf("page fault\t");
}
else
printf(" \t");
printf("%d:\t",page[i]);
for(j=1; j<=nf; j++)
printf(" %d ",fr[j]);
printf("\n");
}
printf("\ntotal page faults: %d",p+3);
}
int full(int a[],int n)
{
int k;
for(k=1; k<=n; k++)
{
if(a[k]==-1)
return 0;
}
}

```

```
}  
return 1;  
}
```

Output:

```
enter the number of frames:5  
enter the page size2  
enter the page sequence:1  
2  
1: 1 -1 -1 -1 -1  
2: 1 2 -1 -1 -1  
total page faults: 3
```

Result:

Thus the program was executed successfully and hence output verified

Ex. 9.a IMPLEMENT DISK SCHEDULING ALGORITHM-FCFS

Date:

Aim:

To write a C program to implement the **First Come First Serve (FCFS) disk scheduling algorithm** and calculate the total seek time.

Algorithm:

1. Start from initial head position.
2. Traverse requests in given order.
3. Calculate seek time = $|\text{current} - \text{next}|$.
4. Add all seek times.

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, seek_time = 0;
    scanf("%d", &n);
    int req[n];
    for(int i = 0; i < n; i++) scanf("%d", &req[i]);
    scanf("%d", &head);
    printf("Seek Sequence: %d", head);
    for(int i = 0; i < n; i++) {
        seek_time += abs(head - req[i]);
        head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Seek Time = %d", seek_time);
    return 0;
}
```

Output:

Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Seek Time = 640

Result:

Thus the program was executed successfully and hence output verified.

Ex.No:9.b

SSTF (SHORTEST SEEK TIME FIRST)

Date:

Aim:

To write a C program to implement the **Shortest Seek Time First (SSTF) disk scheduling algorithm** and calculate the total seek time.

Algorithm:

1. Start from head position.
2. Find closest request.
3. Move head to that request.
4. Mark it visited.
5. Repeat until all requests are served.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, head, seek_time = 0;
    scanf("%d", &n);
    int req[n], visited[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &req[i]);
        visited[i] = 0;
    }
    scanf("%d", &head);
    printf("Seek Sequence: %d", head);
    for(int i = 0; i < n; i++) {
        int min = 9999, index = -1;
        for(int j = 0; j < n; j++) {
            if(!visited[j] && abs(head - req[j]) < min) {
                min = abs(head - req[j]);
                index = j;
            }
        }
        visited[index] = 1;
        seek_time += min;
        head = req[index];
        printf(" -> %d", head);
    }
    printf("\nTotal Seek Time = %d", seek_time);
    return 0;
}
```

Sample Input:

Enter number of requests: 8

Enter request queue:

98 183 37 122 14 124 65 67

Enter initial head position:

53

Output:

Seek Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183

Total Seek Time = 236

Result:

The program was executed successfully and hence output verified

Ex.No.10 .a

FILE ALLOCATION STRATEGY- SEQUENTIAL

Date:

Aim:

To write a C Program for sequential file allocation.

Algorithm:

Step 1: Start the program.

Step 2: Get the starting address and the length of the file

Step 3: Read the values.

Step 4: Assign count value to length

Step 5: Allocate the file in contiguous order till count equals the length value

Step 6: If all the files are assigned in the memory, display file has been allocated Successfully.

Step 7: If any error or memory insufficiency then display file cannot be allocated

Step 8: Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int f[50], i, st, j, len, c, k, count=0;
clrscr();
for(i=0; i<50; i++)
f[i]=0;
printf ("Files Allocated Are:");
x: count=0;
printf ("\n Enter starting block and length of files");
scanf ("%d%d", &st, &len);
for(k=st; k<=(st+len); k++)
if (f[k]==0)
count++;
if(len==count)
{
for(j=st; j<=(st+len); j++)
if(f[j]==0)
{
f[j]=1;
printf("\%d\t%d\n", j, f[j]);
}
if(j!=(st+len-1))
printf(" The file is allocated to disk\n");
```

```

}
else
printf("The file is not allocated\n");
count=0;
printf("\n Do you want to enter more files(Yes - 1/No - 0)");
scanf("%d",&c);
if(c==1)
goto x;
else
exit();
getch();
}

```

Output:

```

Files Allocated are :
Enter starting block and length of files: 14 3
14 1
15 1
16 1
The file is allocated to disk Do you want to enter more file(Yes - 1/No - 0)1
Enter starting block and length of files: 14 1
The file is not allocated
Do you want to enter more file(Yes - 1/No - 0)1
Enter starting block and length of files: 14 4
The file is not allocated
Do you want to enter more file(Yes - 1/No - 0)0

```

Result:

Thus a C Program for sequential file allocation was executed successfully.

Ex.No.10 .b**FILE ALLOCATION STRATEGY – INDEXED****Date:****Aim:**

To write a C Program for indexed file allocation strategy.

Algorithm:

Step 1: Start the program.

Step 2: Get the value for the index.

Step 3: Read the index value.

Step 4: Get the value for block and the file to be allocated.

Step 5: check if the file is already entered.

Step 6: If not allocate the files up to the block number to that index mentioned.

Step 7: Check if there is another block of files to be allocated.

Step 8: If yes continue from step: 2

Step 9: If not end the program.

Step 10: Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
Void main()
{
    int f[50], i, k, j, index[50], n, c, st, len, ind, count=0;
    clrscr();
    for(i=0; i<50; i++)
        f[i]=0;
    x:printf("Enter the index block:");
    scanf("%d",&ind);
    if(f[ind]!=1)
    {
        printf("Enter no of blocks needed and no of files for index %d on the disk:\n", ind);
        scanf("%d",&n);
    }
}
```

```

Else
{
Printf(“%d index is already allocated\n”,ind);
Goto x;
}
y:count=0;
for(i=0; i<n; i++)
{
scanf("%d",&index[i]);
if(f[index[i]]==0)
count++;
}
if(count==n)
{
for(j=0; j<n; j++)
if(index[j])=1;
printf("\nallocated");
printf("\n file indexed");
for(k=0; k<n; k++)
printf("\n%d ----->%d:%d\n", ind, index[k], f[index[k]]);
}
else
{
printf("\n File in the index already allocated");
printf("\nEnter another file indexed");
goto y;
}
printf("\n Do you want to enter one more block(Yes-1/No-0)");
scanf("%d",&c);
if(c==1)
goto x;
else
exit(0);
getch( );
}

```

Output:

```
Enter the index block: 5
Enter no of blocks needed and no of files for the index 5 on the disk : 4
1 2 3 4
Allocated
File Indexed
5----->1: 1
5----->2: 1
5----->3: 1
5----->4: 1
Do you want to enter more file (Yes - 1/No - 0)1
Enter the index block: 4
4 index is already allocated
Enter the index block: 6
Enter no of blocks needed and no of files for the index 6 on the disk : 2
7 8
Allocated
File Indexed
6----->7: 1
6----->8: 1
Do you want to enter more file (Yes - 1/No - 0)0
```

Result:

Thus a C Program for indexed file allocation strategy executed successfully.

Ex no: 10.c

LINKED FILE ALLOCATION

Date:

Aim:

Write a C Program for linked file allocation strategy.

Algorithm:

Step 1: Start the program.

Step 2: Get the values of the block that have been already allocated.

Step 3: Read the values of the block.

Step 4: Get the index value and number of blocks that has to be allocated.

Step 5: Check if the files already exist.

Step 6: If yes, then display file is already allocated.

Step 7: Else allocate the files to the specified index value.

Step 8: Check if there is any other files to be allocated.

Step 9: If yes go to step 2 else end the program.

Step 10: Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
main()
{
    int f[50],p,i,j,k,a,st,len,n;
    for(i=0;i<50;i++)
        f[i]=0;
    printf("enter how many blocks already allocated");
    scanf("%d",&p);
    printf("\nenter the blocks nos");
    for(i=0;i<p;i++)
    {
        scanf("%d",&a);
        f[a]=1;
    }
    X:
    printf("enter index saring block & length");
    scanf("%d%d",&st,&len);
    k=len;
    if(f[st]==0)
    {
        for(j=st;j<(k+st);j++)
        {
            if(f[j]==0)
            {
```

```

        f[j]=1;
printf("\n%d->%d",j,f[j]);
    }
else
{
    printf("\n %d->file is already allocated",j);
    k++;
}
}
}
else
printf("\nif u enter one more (yes-1/no-0)");
scanf("%d",&c);
if(c==1)
goto X;
else
exit();
getch( );
}

```

Input:

```

enter how many blocks already allocated
5
enter block nos
3
7
9
10
14
Enter index starting block & length
4
10

```

Output:

4->1
5->1
6->1
7->file is already allocated
8->1
9->file is already allocated
10->file is already allocated
11->1
12->1
13->1
14->file is already allocated
15->1
16->1
17->1

Result:

The program for file allocation technique- linked file allocation strategy has been executed and shown the results as above.